

Get Started with Jenkins 2.0: Pipeline as Code

With the Pipeline suite of plugins, the mantra of Jenkins 2.0—of ‘Pipeline as Code’—is gaining momentum. This is evident from the growing number of organisations embracing this new way of codifying build flow. This article’s objective is to lower the entry barrier for the DevOps and engineering teams that are at the threshold of switching over to Jenkins 2.0 and Pipeline as Code, and help them get started.

For many years, Jenkins has been the most preferred choice when it comes to practising automated continuous integration. The traditional (also known as freestyle) way of setting up new projects in Jenkins has been (and still is) through the graphical user interface. With the help of the GUI, pre- and post-build tasks, including handling parameters and injecting custom environment variables, can be glued together in a single project.

With the release of Jenkins 2.0, there has been a shift in the strategy to set up new projects in Jenkins. Almost all the build tasks are treated as Pipeline as Code, using the suite of Pipeline plugins, which means that the complete build flow can be codified. Therefore, it can be tracked in version control, thus enabling collaboration across teams. For a quick overview and highlights of the features in Jenkins 2.0, visit <https://jenkins.io/2.0/>.

Jenkinsfile is where Pipeline as Code is maintained, and it should be placed in the root directory of the source code.

Partitioning CI/CD (Continuous Integration/Continuous Deployment) into logically unique but interconnected parts, and the visual representation of this entire chain of events, is termed as build flow and this is what makes Pipeline attractive.

Pipeline is gradually evolving into the new way to programmatically construct a logical flow of tasks—for example, compile source code, generate artifacts and deploy them—all by using a variety of plugins and third-party tools. However, it should be emphasised that the build system (*make*, *ant*, or a mix of scripts in whatever language you

choose) does not need to be revamped and can be reused as is.

Prerequisites or assumptions

Before we proceed further, make sure you have access to Jenkins (v2.3x and above) with standard plugins, and are able to set up a test project. You should also have the basic familiarity to set up new projects in Jenkins.

This article will be based on the syntax of scripted Pipeline, a general-purpose DSL that’s based on the Groovy programming language. However, the declarative syntax can be used too to achieve the same results with minor modifications.

The definitive place to start reading about Jenkins Pipeline is <https://jenkins.io/doc/book/pipeline/>.

The link <https://jenkins.io/doc/pipeline/steps/> is the reference for various steps supported in the Pipeline environment.

A comparison of the two syntaxes (imperative and declarative) is documented at <https://jenkins.io/doc/book/pipeline/syntax/#compare>.

Anatomy

Let’s begin with analysing Pipeline as Code by focusing on the individual stages, one at a time. Groovy code runs in a JVM environment in a build agent (not called slaves any more) and in a tightly controlled environment, i.e., a Groovy sandbox. This sandbox environment ensures that Pipeline as Code cannot be used to invoke system calls. These calls are made available by Jenkins for administrative purposes—so use them with caution!

An easy and quick reference on how to auto-generate code snippets for most of the Pipeline steps is available with

